EXPERIMENT- 9

Aim: To perform various techniques for testing using the testing tool for <case study name>

Theory:

1) What is Testing and its importance in Software development.

What is testing?

Testing is the process of evaluating a system or its component(s) with the intent to find that whether it satisfies the specified requirements or not. This activity results in the actual, expected and difference between their results. In simple words testing is executing a system in order to identify any gaps, errors or missing requirements in contrary to the actual desire or requirements.

According to ANSI/IEEE 1059 standard, Testing can be defined as *A process of analyzing a software item to detect the differences between existing and required conditions (that is defects/errors/bugs) and to evaluate the features of the software item*.

Who does testing?

It depends on the process and the associated stakeholders of the project(s). In the IT industry, large companies have a team with responsibilities to evaluate the developed software in the context of the given requirements. Moreover, developers also conduct testing which is called Unit Testing. In most cases, following professionals are involved in testing of a system within their respective capacities:

- Software Tester
- Software Developer
- Project Lead/Manager
- End User

Different companies have difference designations for people who test the software on the basis of their experience and knowledge such as Software Tester, Software Quality Assurance Engineer, and QA Analyst etc.

It is not possible to test the software at any time during its cycle. The next two sections state when testing should be started and when to end it during the SDLC.

When to Start Testing?

An early start to testing reduces the cost, time to rework and error free software that is delivered to the client. However in Software Development Life Cycle (SDLC) testing can be started from the Requirements Gathering phase and lasts till the deployment of the software. However it also depends on the development model that is being used. For example in Water fall model formal testing is conducted in the Testing phase, but in incremental model, testing is performed at the end of every increment/iteration and at the end the whole application is tested.

Testing is done in different forms at every phase of SDLC like during Requirement gathering phase, the analysis and verifications of requirements are also considered testing. Reviewing the design in the design phase with intent to improve the design is also considered as testing. Testing performed by a developer on completion of the code is also categorized as Unit type of testing.

- Management decision.

Verification & Validation

These two terms are very confusing for people, who use them interchangeably. Let's discuss about them briefly.

S.N.	Verification	Validation
1	Are you building it right?	Are you building the right thing?
2	Ensure that the software system meets all the functionality.	Ensure that functionalities meet the intended behavior.

3	Verification takes place first and includes the checking for documentation, code etc.	Validation occurs after verification and mainly involves the checking of the overall product.
4	Done by developers.	Done by Testers.
5	Have static activities as it includes the reviews, walkthroughs, and inspections to verify that software is correct or not.	Have dynamic activities as it includes executing the software against the requirements.
6	It is an objective process and no subjective decision should be needed to verify the Software.	It is a subjective process and involves subjective decisions on how well the Software works.

2) Different types of Testing.

There are many types of testing like

- Unit Testing
- Integration Testing
- Functional Testing
- System Testing
- Stress Testing
- Performance Testing
- Usability Testing
- Acceptance Testing
- Regression Testing
- Beta Testing

How to write test cases?

Here is a simple test case format

Fields in test cases:

Test case id:

Unit to test: What to be verified?

Assumptions:

Test data: Variables and their values

Steps to be executed:

Expected result:

Actual result:

Pass/Fail:

Comments:

Procedure:

1) Identify various modules of the system so that they can be tested stand alone.

2) Identify the groups of the module that can be tested together

3) Perform the testing of the modules as a unit .

Output: Development of upto four test cases and their applied results on the system designed

PART-B

Name	Eklavya Vail	Aarush Mishra	Mithas Jain
Roll no.	B117	B155	B160

Test Case 1: Valid User Authentication

- **Test case id:** TC_01
- **Unit to test:** User Authentication & Login Module (Verification of Role-Based Access)
- **Assumptions:** The user is registered in the database with the 'Patient' role.
- **Test data:** UserID = "pat_123", SecretPasscode = "securePass!@#"
- **Steps to be executed:**
    1. Navigate to the Aegis Link Login Page.
    2. Enter UserID and SecretPasscode.
    3. Click the 'Sign In' button.
- **Expected result:** The system validates the credentials, matches the ID/Passcode, and securely routes the user to the Patient Dashboard.
- **Actual result:** User is routed to the Patient Dashboard.
- **Pass/Fail:** Pass
- **Comments:** The IDPasscodeMatches() function operates correctly.

Test Case 2: Invalid User Authentication

- **Test case id:** TC_02
- **Unit to test:** User Authentication & Login Module (Negative Testing)

- **Assumptions:** The user exists, but inputs incorrect login credentials.
- **Test data:** UserID = "pat_123", SecretPasscode = "wrongPass123"
- **Steps to be executed:**
    1. Navigate to the Login Page.
    2. Enter UserID.
    3. Enter the incorrect SecretPasscode.
    4. Click 'Sign In'.
- **Expected result:** The system denies access, invokes the IfInvalid() loop, and displays an "Invalid Credentials" error message.
- **Actual result:** Access denied, "Invalid Credentials" displayed.
- **Pass/Fail:** Pass
- **Comments:** Security validation successfully prevents unauthorized access.

Test Case 3: Log Daily Vitals (Valid Input)

- **Test case id:** TC_03
- **Unit to test:** HealthTracking Management
- **Assumptions:** The patient is successfully authenticated and logged into the platform.
- **Test data:** BloodPressure = "120/80", HeartRate = 75, Temperature = 98.6
- **Steps to be executed:**
    1. Open the 'Log Vitals' interface.
    2. Input the test data values into the respective fields.
    3. Click 'Submit Vitals'.
- **Expected result:** The system validates the parameters, saves the record to the PostgreSQL database, and displays a success confirmation prompt.
- **Actual result:** Record saved, confirmation prompt displayed.
- **Pass/Fail:** Pass
- **Comments:** Basic data entry and integrity constraints are working.

Test Case 4: Anomaly Detection and Emergency Alert

- **Test case id:** TC_04
- **Unit to test:** AlertEngine / HealthController (Dynamic state machine testing)
- **Assumptions:** The patient is logged in, and a Caretaker is actively assigned to their profile.
- **Test data:** HeartRate = 160 (Threshold max = 120)
- **Steps to be executed:**
    1. Open the 'Log Vitals' interface.
    2. Input an abnormally high HeartRate of 160.
    3. Click 'Submit Vitals'.
- **Expected result:** The system evaluates the thresholds, detects the anomaly, and dispatches a High-Priority Push Notification to the assigned Caretaker's dashboard.
- **Actual result:** Alert triggered and pushed to Caretaker interface.
- **Pass/Fail:** Pass
- **Comments:** Automated anomaly response meets intended behavior parameters.

Test Case 5: Gender-Specific UI Logic

- **Test case id:** TC_05
- **Unit to test:** Patient Dashboard UI Logic
- **Assumptions:** A male patient logs into the system.
- **Test data:** User Gender = "Male"
- **Steps to be executed:**
    1. Authenticate with a male patient account.
    2. Load the Patient Dashboard.
    3. Scan the UI for the Reproductive Health module.

- **Expected result:** The system evaluates the "Is Patient Female?" condition check as 'No' and hides/disables the Menstrual & Reproductive Health tracking module.
- **Actual result:** Module is hidden from the dashboard view.
- **Pass/Fail:** Pass
- **Comments:** UI successfully adapts dynamically based on user parameters.

Test Case 6: Caretaker Medication Management

- **Test case id:** TC_06
- **Unit to test:** Medication Management Module
- **Assumptions:** A caretaker is logged in and possesses write access to the patient's schedule.
- **Test data:** PatientID = "pat_123", MedName = "Aspirin", NewDosage = "100mg"
- **Steps to be executed:**
    1. Navigate to the Caretaker Dashboard.
    2. Open 'Manage Medications' for pat_123.
    3. Edit the dosage of Aspirin to '100mg' and click 'Update'.
- **Expected result:** The dosage is updated in the database, an audit trail entry is generated, and a dose update notification is queued for the patient.
- **Actual result:** Dosage successfully updated, audit logged.
- **Pass/Fail:** Pass
- **Comments:** Caretaker oversight features modify patient records correctly.